

Unidad 4

Introducción a Java

Tabla de contenido

1. Introducción.....	4
1.1. Generando un programa en Java.....	4
1.2. La Máquina Virtual de Java (JVM).....	5
2. Conceptos de Programación Orientada a Objetos	5
2.1. ¿Qué es un objeto?	6
2.2. ¿Qué es una clase?	8
2.3. ¿Qué es la herencia?	9
2.4. Exercises	11
3. Fundamentos del lenguaje	11
3.1. variables	11
3.2. Tipos de datos primitivos y Strings	12
3.2.1. Valores por defecto	13
3.3. Literales	14
3.4. Sentencias de control de flujo.....	14
3.4.1 if-then	14
3.4.2. if-then-else.....	15
3.4.3. switch	16
3.4.4. while y do-while	21
3.4.5. for	22
3.4.6. Instrucciones break, continue y return	24
3.5. Arrays.....	26
3.5.1. Declarando un Array	28
3.5.2. Creación, inicialización y acceso a un array.	29
3.5.3. Copiando Arrays.....	30
3.5.4. Manipulación de arrays	31
3.5.5. Ejercicios con Arrays.....	32
3.5.6. Ordenando Arrays	33
3.5.7. Acceder a elementos de un array con un bucle For.....	34

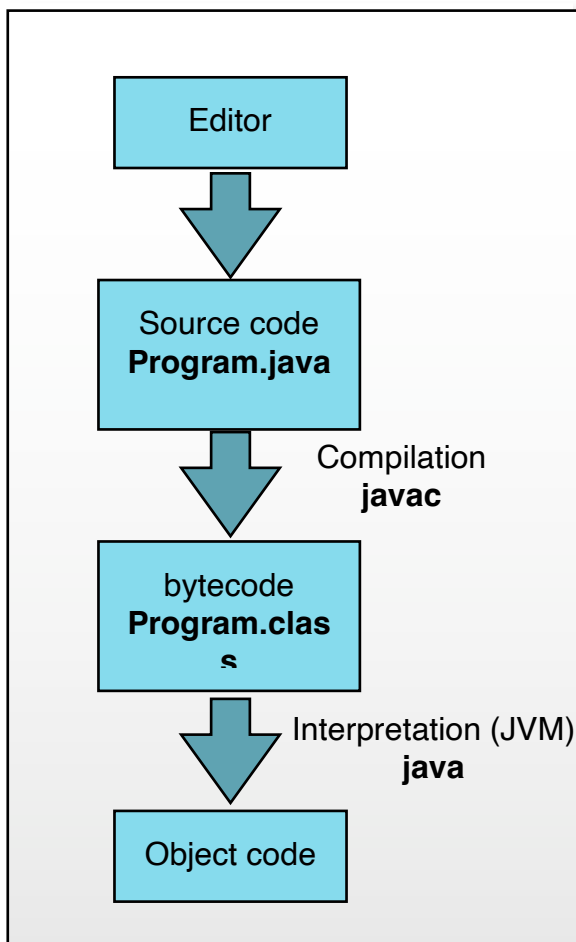
1. Introducción.

Java es un lenguaje de propósito general, y es adecuado para desarrollar cualquier tipo de aplicaciones profesionales.

Las principales características del lenguaje son:

- Es puramente Orientado a Objetos.
- Funciona perfectamente en red.
- Utiliza muchas de las características de los lenguajes modernos evitando sus desventajas.
- Tiene una gran funcionalidad gracias a sus librerías.
- NO tiene punteros manejados directamente por el programador, aunque los implementa internamente.
- La gestión de memoria no es un "problema". Es manejada por el sistema automáticamente.
- Genera programas con pocos errores posibles.
- Tiene multi-Threading (tareas paralelas ejecutándose a la vez).

Puede ser considerado como una evolución de C++ porque la sintaxis es similar, pero fue diseñado desde cero.



1.1. Generando un programa en Java

¿Es Java un lenguaje compilado o interpretado?

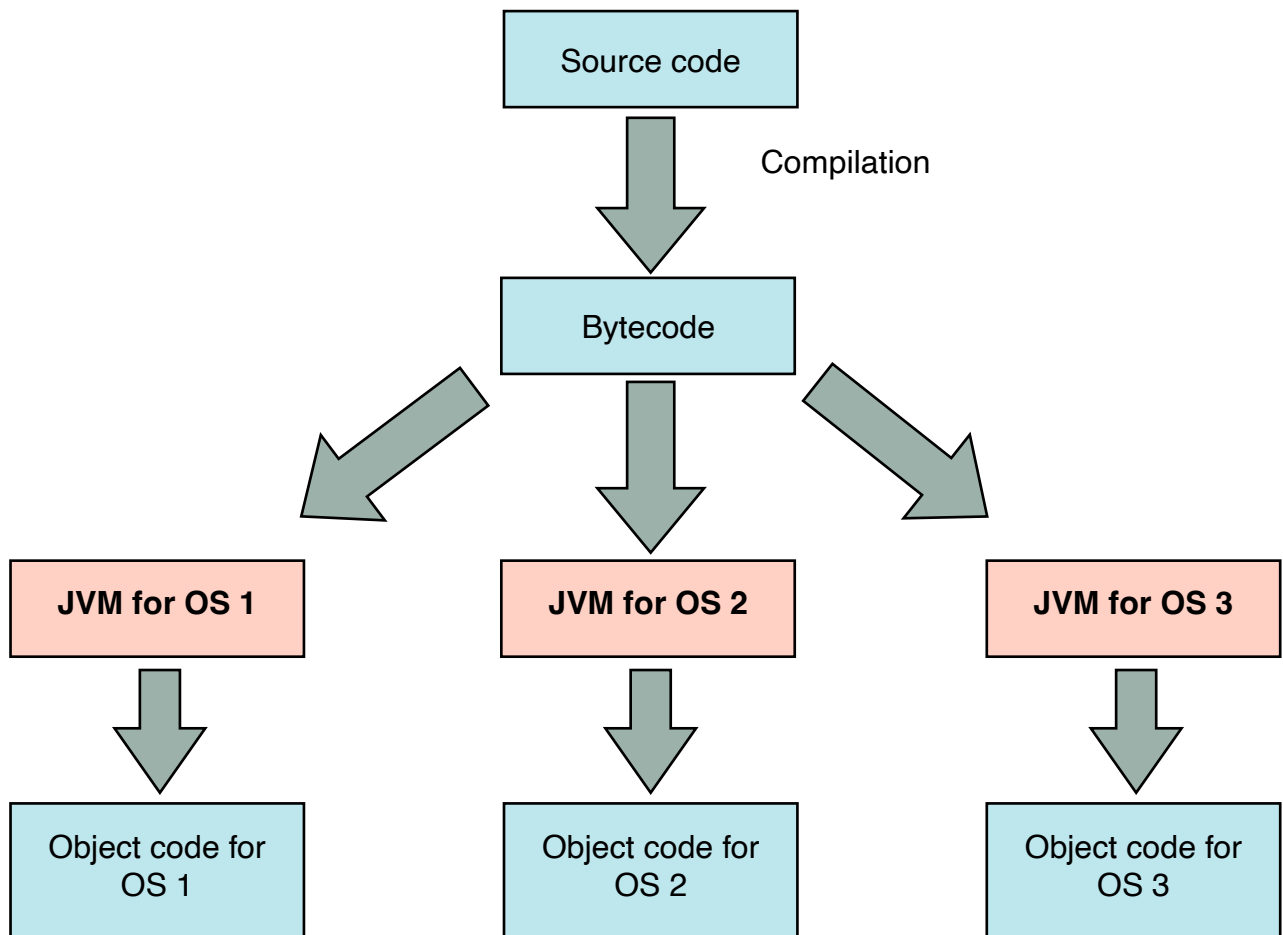
Ni una cosa ni la otra. Estrictamente hablando es interpretado, pero necesita un proceso previo de compilación. Una vez que el programa es compilado, se genera un fichero usando el llamado "bytecode" o `j_code`. Para ejecutar el programa se necesita un intérprete, la JVM o Máquina Virtual de Java. De esta manera es posible compilar un programa y ejecutarlo en diferentes Sistemas Operativos (usando la JVM específica para cada uno de ellos).

El hecho de que Java sea interpretado no afecta demasiado a la velocidad de los programas porque el bytecode es casi código objeto. También ayuda el uso de compiladores JIT (Just In Time) que traducen el bytecode a código objeto.

Los programas en Java son robustos y tienen pocos errores, básicamente por el manejo automático de memoria y punteros.

1.2. La Máquina Virtual de Java (JVM)

La JVM es la idea revolucionaria del lenguaje. Es la entidad que proporciona independencia de plataforma para los programas Java compilados a **bytecode**.



El mismo programa de código fuente compilado en diferentes plataformas o Sistemas Operativos genera el mismo fichero de bytecode. Esto es lógico porque el compilador de Java traduce el código a código objeto para una máquina virtual (la JVM).

Si ese programa de bytecode puede ser ejecutado en diferentes plataformas o Sistemas Operativos es porque hay un traductor de ese bytecode a código nativo para ese Sistema Operativo.

2. Conceptos de Programación Orientada a Objetos

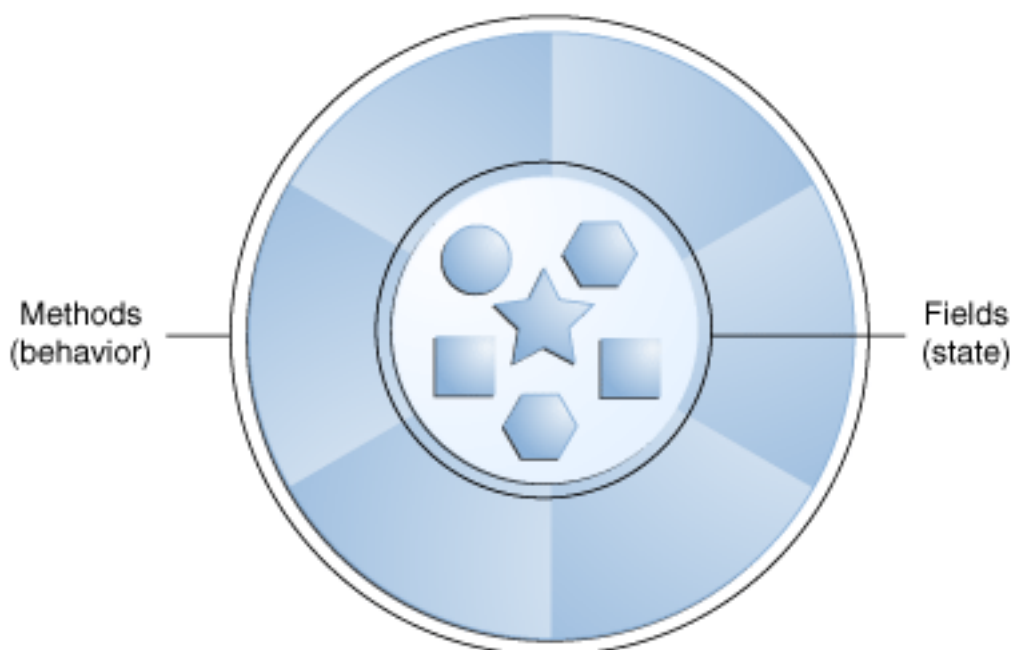
La POO (Programación Orientada a Objetos) es una evolución de la programación estructurada o tradicional. Utiliza objetos para crear un programa. Estos objetos consisten en una serie de características y operaciones que se pueden realizar sobre ellos. Estos objetos no están aislados en el programa, sino que se comunican entre sí.

2.1. ¿Qué es un objeto?

Los objetos son clave para comprender la tecnología orientada a objetos. Observa a tu alrededor y encontrarás muchos ejemplos de objetos del mundo real: tu perro, tu escritorio, tu televisor, tu bicicleta.

Los objetos del mundo real comparten dos características: todos tienen estado y comportamiento. Los perros tienen estado (nombre, color, raza, hambre) y comportamiento (ladrar, buscar, mover la cola). Las bicicletas también tienen estado (marcha actual, cadencia de pedaleo actual, velocidad actual) y comportamiento (cambiar de marcha, cambiar de cadencia de pedaleo, frenar). Identificar el estado y el comportamiento de los objetos del mundo real es una excelente manera de comenzar a pensar en términos de programación orientada a objetos.

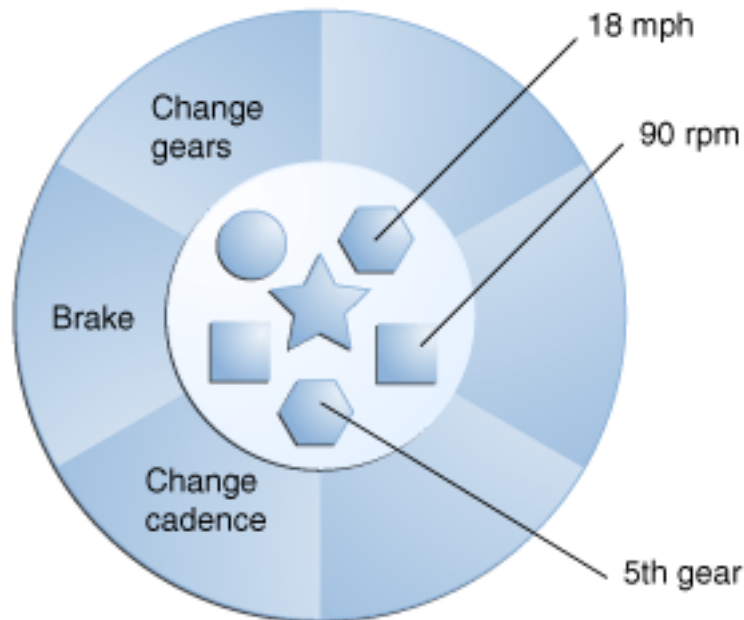
Tómate un minuto ahora mismo para observar los objetos del mundo real que se encuentran a tu alrededor. Para cada objeto que veas, hazte dos preguntas: "¿En qué estados posibles puede estar este objeto?" y "¿Qué comportamiento posible puede realizar este objeto?". Asegúrate de anotar tus observaciones. Al hacerlo, notarás que la complejidad de los objetos del mundo real varía. Su lámpara de escritorio puede tener solo dos estados posibles (encendida y apagada) y dos comportamientos posibles (encender, apagar), pero su radio de escritorio podría tener estados adicionales (encendido, apagado, volumen actual, emisora actual) y comportamientos adicionales (encender, apagar, subir volumen, bajar volumen, buscar, escanear y sintonizar). También puede observar que algunos objetos, a su vez, contienen otros objetos. Todas estas observaciones del mundo real se traducen al mundo de la programación orientada a objetos.



Los objetos de software son conceptualmente similares a los objetos del mundo real: también constan de estado y comportamiento relacionado. Un objeto almacena su estado en campos (variables en algunos lenguajes de programación) y expone su comportamiento mediante métodos (funciones en otros lenguajes de programación). Los

métodos operan sobre el estado interno de un objeto y sirven como mecanismo principal para la comunicación entre objetos. Ocultar el estado interno y exigir que toda interacción se realice a través de los métodos de un objeto se conoce como encapsulación de datos, un principio fundamental de la programación orientada a objetos.

Considera una bicicleta, por ejemplo:



Una bicicleta modelada como un objeto de software.

Al atribuir el estado (velocidad actual, cadencia de pedaleo actual y marcha actual) y proporcionar métodos para cambiar dicho estado, el objeto mantiene el control sobre cómo el mundo exterior puede usarlo. Por ejemplo, si la bicicleta solo tiene 6 marchas, un método para cambiarlas podría rechazar cualquier valor menor que 1 o mayor que 6. Agrupar código en objetos de software individuales ofrece diversas ventajas, entre ellas:

- **Modularidad:** El código fuente de un objeto puede escribirse y mantenerse independientemente del código fuente de otros objetos. Una vez creado, un objeto puede distribuirse fácilmente dentro del sistema.
- **Ocultación de información:** Al interactuar únicamente con los métodos de un objeto, los detalles de su implementación interna permanecen ocultos al mundo exterior.
- **Reutilización de código:** Si un objeto ya existe (quizás escrito por otro desarrollador de software), puede usarlo en su programa. Esto permite a los especialistas implementar, probar y depurar objetos complejos y específicos para cada tarea, que luego puede confiar en que se ejecutarán en su propio código.
- **Conectabilidad y facilidad de depuración:** Si un objeto en particular resulta problemático, simplemente puede eliminarlo de su aplicación e instalar otro objeto como reemplazo. Esto es similar a solucionar problemas mecánicos en el mundo real. Si un componente se rompe, se reemplaza, no toda la máquina.

2.2. ¿Qué es una clase?

En el mundo real, a menudo encontrarás muchos objetos individuales del mismo tipo. Puede haber miles de otras bicicletas en existencia, todas de la misma marca y modelo. Cada bicicleta fue construida a partir del mismo conjunto de planos y, por lo tanto, contiene los mismos componentes. En términos orientados a objetos, decimos que tu bicicleta es una **instancia** de la **clase** de objetos conocida as bicicletas. Una **clase es el plano a partir del cual se crean los objetos individuales**.

En POO **Instancia** y **Objeto** son sinónimos.

La siguiente clase Bicycle es una posible implementación de una bicicleta:

```
class Bicycle {
    int cadence = 0;
    int speed = 0;
    int gear = 1;

    void changeCadence(int newValue) {
        cadence = newValue;
    }

    void changeGear(int newValue) {
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
}
```

La sintaxis del lenguaje de programación Java te parecerá nueva, pero el diseño de esta clase se basa en la discusión previa sobre los objetos de bicicleta. Los campos `cadence`, `speed` y `gear` representan el estado del objeto, y los métodos (`changeCadence`, `changeGear`, `speedUp`, etc.) definen su interacción con el mundo exterior.

Quizás hayas notado que la clase `Bicycle` no contiene un método principal. Esto se debe a que no es una aplicación completa; es solo el plano de las bicicletas que podrían usarse en una aplicación. La responsabilidad de crear y usar nuevos objetos `Bicycle` recae en otra clase de tu aplicación.

Aquí tienes una clase `BicycleDemo` que crea dos objetos `Bicycle` independientes e invoca sus métodos:

```
class BicycleDemo {
    public static void main(String[] args) {

        // Create two different
        // Bicycle objects
        Bicycle bike1 = new Bicycle();
        Bicycle bike2 = new Bicycle();

        // Invoke methods on
        // those objects
        bike1.changeCadence(50);
        bike1.speedUp(10);
        bike1.changeGear(2);
        bike1.printStates();

        bike2.changeCadence(50);
        bike2.speedUp(10);
        bike2.changeGear(2);
        bike2.changeCadence(40);
        bike2.speedUp(10);
        bike2.changeGear(3);
        bike2.printStates();
    }
}
```

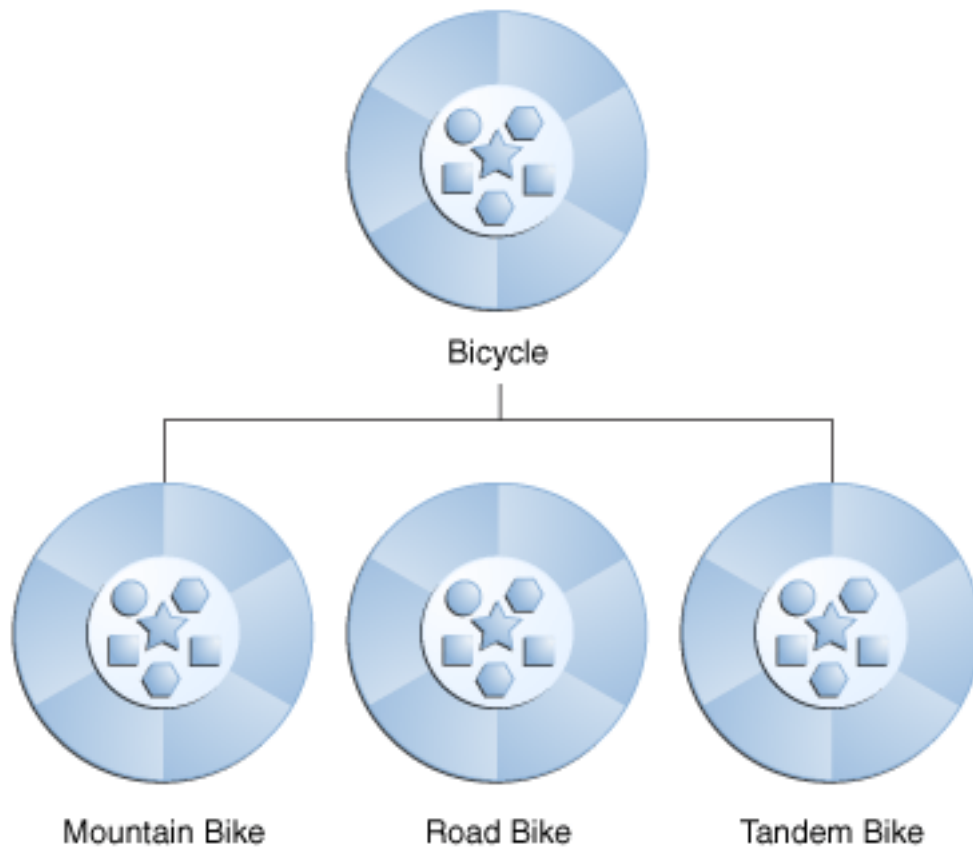
La salida de esta prueba imprime la cadencia de pedaleo, la velocidad y la marcha finales para las dos bicicletas:

```
cadence:50 speed:10 gear:2
cadence:40 speed:20 gear:3
```

2.3. ¿Qué es la herencia?

Diferentes tipos de objetos a menudo tienen una cierta cantidad de estados y comportamientos en común. Las bicicletas de montaña, las de carretera y los tandems, por ejemplo, comparten las características de las bicicletas (velocidad actual, cadencia de pedaleo actual, marcha actual). Sin embargo, cada uno también define características adicionales que los hacen diferentes.

La programación orientada a objetos permite que las clases *hereden* estados y comportamientos de uso común de otras clases. En este ejemplo, Bicycle ahora se convierte en la **superclase** de MountainBike, RoadBike y TandemBike. En el lenguaje de programación Java, cada clase puede tener una superclase directa, y cada superclase tiene el potencial de un número ilimitado de **subclases**.



A hierarchy of bicycle classes.

La sintaxis para crear una subclase es simple. Al principio de la declaración de tu clase, usa la palabra clave **extends**, seguida del nombre de la clase de la que se va a heredar:

```
class MountainBike extends Bicycle {  
    // new fields and methods defining  
    // a mountain bike would go here  
}
```

Esto proporciona a **MountainBike** los mismos campos y métodos que **Bicycle**, pero permite que su código se centre exclusivamente en las características que la hacen única. Esto facilita la lectura del código de las subclases. Sin embargo, debe asegurarse de

documentar correctamente el estado y el comportamiento que define cada superclase, ya que ese código no aparecerá en el archivo fuente de cada subclase.

2.4. Exercises

Puedes realizar ejercicios en esta URL (sólo las preguntas del 1 al 6. El resto lo veremos en el siguiente tema):

<http://docs.oracle.com/javase/tutorial/java/concepts/QandE/questions.html>

3. Fundamentos del lenguaje

3.1. variables



El lenguaje de programación Java define los siguientes tipos de variables:

- **Variables de Instancia** (Atributos no estáticos) Técnicamente, los objetos almacenan sus estados individuales en "atributos no estáticos", es decir, atributos declarados sin la palabra clave `static`. Los campos no estáticos también se conocen como variables de instancia porque sus valores son únicos para cada instancia de una clase (es decir, para cada objeto); la velocidad actual de una bicicleta es independiente de la velocidad actual de otra.
- **Variables de Clase** (Atributos estáticos) Una variable de clase es cualquier campo declarado con el modificador `static`; esto indica al compilador que existe exactamente una copia de esta variable, independientemente de cuántas veces se haya instanciado la clase. Un atributo que define el número de marchas para un tipo particular de bicicleta podría marcarse como estático, ya que conceptualmente se aplicará el mismo número de marchas a todas las instancias. El código `static int numGears = 6;` crearía dicho atributo estático. Además, se podría añadir la palabra clave `final` para indicar que el número de marchas nunca cambiará.
- **Variables locales.** De forma similar a cómo un objeto almacena su estado en atributos, un método suele almacenar su estado temporal en **variables locales**. La sintaxis para declarar una variable local es similar a la de un atributo (por ejemplo, `int count = 0;`). No existe una palabra clave especial que designe una variable como local; dicha determinación se basa completamente en la ubicación donde se declara la variable, que se encuentra entre las llaves de apertura y cierre de un método. Por lo tanto, las variables locales solo son visibles para los métodos en los que se declaran; no son accesibles desde el resto de la clase.
- **Parámetros.** Ya ha visto ejemplos de parámetros, tanto en la clase `Bicycle` como en el método principal de la aplicación "¡Hola Mundo!". Recuerda que la declaración del método principal es `public static void main(String[] args)`. Aquí, la variable `args` es el parámetro de este método. Es importante recordar que los parámetros siempre se clasifican como "variables", no como "atributos". Esto también se aplica a otras construcciones que aceptan parámetros (como constructores y manejadores de excepciones) que aprenderás más adelante en el tema.

3.2. Tipos de datos primitivos y Strings

En el lenguaje de programación Java todas las variables deben declararse antes de poder usarse. Esto implica indicar el tipo y el nombre de la variable, como ya se ha visto:

```
int gear = 1;
```

De esta forma, se le indica al programa que existe un campo llamado "gear", que contiene datos numéricos y tiene un valor inicial de "1". El tipo de dato de una variable determina los valores que puede contener, así como las operaciones que se pueden realizar con ella. Además de int, Java admite otros siete tipos de datos primitivos. Un tipo primitivo está predefinido por el lenguaje y se nombra mediante una palabra clave reservada. Los ocho tipos de datos primitivos que admite Java son:

- **byte**: El tipo de dato "byte" es un entero de 8 bits con signo en complemento a dos. Tiene un valor mínimo de -128 y un valor máximo de 127 (ambos inclusive). El tipo de dato "byte" puede ser útil para ahorrar memoria en arrays grandes, donde el ahorro de memoria es realmente importante. También se puede usar en lugar de "int" cuando sus límites ayudan a aclarar el código; el hecho de que el rango de una variable sea limitado puede servir como documentación.
- **short**: El tipo de dato "short" es un entero de 16 bits con signo en complemento a dos. Tiene un valor mínimo de -32,768 y un valor máximo de 32,767 (ambos inclusive). Al igual que con "byte", se aplican las mismas directrices: se puede usar "short" para ahorrar memoria en arrays grandes, donde el ahorro de memoria es realmente importante.
- **int**: El tipo de dato "int" es un entero de 32 bits con signo en complemento a dos. Tiene un valor mínimo de -2.147.483.648 y un valor máximo de 2.147.483.647 (inclusive). Para valores enteros, este tipo de dato suele ser la opción predeterminada, a menos que exista una razón (como la anterior) para elegir otra. Este tipo de dato probablemente será lo suficientemente grande para los números que usará su programa, pero si necesita un rango de valores más amplio, utilice long.
- **long**: El tipo de dato long es un entero con signo en complemento a dos de 64 bits. Tiene un valor mínimo de -9.223.372.036.854.775.808 y un valor máximo de 9.223.372.036.854.775.807 (inclusive). Utilice este tipo de dato cuando necesite un rango de valores más amplio que el proporcionado por int.
- **float**: El tipo de dato float es de coma flotante IEEE 754 de 32 bits de precisión simple. Su rango de valores queda fuera del alcance de esta explicación, pero se especifica en la sección "Tipos, formatos y valores de coma flotante" de la Especificación del lenguaje Java. Al igual que con las recomendaciones para byte y short, utiliza float (en lugar de double) si necesitas ahorrar memoria en arrays grandes de números de coma flotante. Este tipo de dato nunca debe utilizarse para

valores precisos, como la moneda. Para ello, deberá utilizar la clase `java.math.BigDecimal`. Números y cadenas abarca `BigDecimal` y otras clases útiles proporcionadas por la plataforma Java.

- **double**: El tipo de dato `double` es coma flotante IEEE 754 de 64 bits y doble precisión. Su rango de valores queda fuera del alcance de esta explicación, pero se especifica en la sección "Tipos, formatos y valores de coma flotante" de la Especificación del lenguaje Java. Para valores decimales, este tipo de dato suele ser la opción predeterminada. Como se mencionó anteriormente, este tipo de dato nunca debe utilizarse para valores precisos, como la moneda.
- **boolean**: El tipo de dato booleano solo tiene dos valores posibles: verdadero y falso. Úsalo para indicadores simples que registren condiciones de verdadero/falso. Este tipo de dato representa un bit de información, pero su tamaño no está definido con precisión.
- **char**: El tipo de dato `char` es un único carácter Unicode de 16 bits. Su valor mínimo es `'\u0000'` (o 0) y su valor máximo es `'\uffff'` (o 65,535 inclusive).

Además de los ocho tipos de datos primitivos mencionados anteriormente, el lenguaje de programación Java también ofrece compatibilidad especial con cadenas de caracteres mediante la clase `java.lang.String`. Al encerrar la cadena de caracteres entre comillas dobles, se creará automáticamente un nuevo objeto `String`; por ejemplo, `String s = "esto es una cadena";`. Los objetos `String` son inmutables, lo que significa que, una vez creados, sus valores no se pueden modificar. La clase `String` no es técnicamente un tipo de datos primitivo, pero considerando la compatibilidad especial que le brinda el lenguaje, probablemente tenderá a considerarla como tal.

3.2.1. Valores por defecto

No siempre es necesario asignar un valor al declarar un campo. El compilador establecerá un valor predeterminado razonable para los campos declarados pero no inicializados. Generalmente, este valor predeterminado será cero o nulo, según el tipo de dato. Sin embargo, **confiar en estos valores predeterminados suele considerarse un mal estilo de programación.**

La siguiente tabla resume los valores predeterminados para los tipos de datos mencionados.

Tipo de datos	Valor por defecto (para atributos)
byte	0
short	0
int	0
long	0L
float	0.0f

double	0.0d
char	\u0000'
String (or any object)	null
boolean	false

Las variables locales son ligeramente diferentes; el compilador nunca asigna un valor predeterminado a una variable local no inicializada. Si no puede inicializar su variable local donde está declarada, asegúrese de asignarle un valor antes de intentar usarla. Acceder a una variable local no inicializada generará un error de compilación.

3.3. Literales

Un *literal* es la representación de un valor fijo en el código fuente; los literales se representan directamente en el código sin necesidad de cálculos. Como se muestra a continuación, es posible asignar un literal a una variable de tipo primitivo:

```
boolean result = true;
char capitalC = 'C';
byte b = 100;
short s = 10000;
int i = 100000;
// The number 26, in decimal
int decVal = 26;
// The number 26, in hexadecimal
int hexVal = 0x1a;
double d1 = 123.4;
// same value as d1, but in scientific notation
double d2 = 1.234e2;
float f1 = 123.4f;
```

3.4. Sentencias de control de flujo

Las sentencias dentro de los archivos fuente generalmente se ejecutan de arriba a abajo, en el orden en que aparecen. Sin embargo, las sentencias de flujo de control interrumpen el flujo de ejecución mediante la toma de decisiones, bucles y ramificaciones, lo que permite que el programa ejecute condicionalmente bloques de código específicos. Esta sección describe las sentencias de toma de decisiones (if-then, if-then-else, switch), las sentencias de bucle (for, while, do-while) y las sentencias de ramificación (break, continue, return) compatibles con el lenguaje de programación Java.

3.4.1 if-then

La sentencia if-then es la más básica de todas las sentencias de flujo de control. Indica al programa que ejecute una sección de código solo si una prueba específica se evalúa como verdadera. Por ejemplo, la clase Bicycle podría permitir que los frenos reduzcan la

velocidad de la bicicleta solo si esta ya está en movimiento. Una posible implementación del método `applyBrakes` podría ser la siguiente:

```
void applyBrakes() {
    // the "if" clause: bicycle must be moving
    if (isMoving){
        // the "then" clause: decrease current speed
        currentSpeed--;
    }
}
```

Si esta prueba se evalúa como falsa (lo que significa que la bicicleta no está en movimiento), el control salta al final de la sentencia `if-then`.

Además, las llaves de apertura y cierre son opcionales, siempre que la cláusula `"then"` contenga solo una sentencia:

```
void applyBrakes() {
    // same as above, but without braces
    if (isMoving)
        currentSpeed--;
}
```

Decidir cuándo omitir las llaves es cuestión de gustos. Omitirlas puede hacer que el código sea más frágil. Si posteriormente se añade una segunda sentencia a la cláusula `"then"`, un error común sería olvidar añadir las nuevas llaves requeridas. El compilador no puede detectar este tipo de error; simplemente se obtendrán resultados erróneos.

3.4.2. if-then-else

La sentencia `if-then-else` proporciona una ruta de ejecución secundaria cuando una cláusula `"if"` se evalúa como falsa. Se podría usar una sentencia `if-then-else` en el método `applyBrakes` para tomar alguna acción si se aplican los frenos cuando la bicicleta no está en movimiento. En este caso, la acción consiste simplemente en imprimir un mensaje de error que indica que la bicicleta ya se ha detenido.

```
void applyBrakes() {
    if (isMoving) {
        currentSpeed--;
    } else {
        System.err.println("The bicycle has already stopped!");
    }
}
```

El siguiente programa, `IfElseDemo`, asigna una calificación según el valor de una prueba: una A para una puntuación del 90 % o superior, una B para una puntuación del 80 % o superior, y así sucesivamente. Utiliza cláusulas `if-then-else` anidadas:

```
import java.util.Scanner;
class IfElseDemo {
    public static void main(String[] args) {
```

```
int testScore;
char grade;
//Reading the value
Scanner inputValue;
inputValue = new Scanner(System.in);
testScore = inputValue.nextFloat();

if (testscore >= 90) {
    grade = 'A';
} else {
    if (testscore >= 80) {
        grade = 'B';
    } else {
        if (testscore >= 70) {
            grade = 'C';
        } else {
            if (testscore >= 60) {
                grade = 'D';
            } else {
                grade = 'F';
            }
        }
    }
}
System.out.println("Grade = " + grade);
}
```

Es posible que hayas notado que el valor de la puntuación de la prueba puede satisfacer más de una expresión en la declaración compuesta: $76 \geq 70$ y $76 \geq 60$. Sin embargo, una vez que se satisface una condición, se ejecutan las declaraciones apropiadas (calificación = 'C'); y las condiciones restantes no se evalúan.

3.4.3. switch

A diferencia de las sentencias if-then y if-then-else, la sentencia switch puede tener varias rutas de ejecución posibles. Una sentencia switch funciona con los tipos de datos primitivos byte, short, char e int. También funciona con tipos enumerados (que se tratan más adelante en Tipos de Enumeración), la clase String y algunas clases especiales que encapsulan ciertos tipos primitivos: Character, Byte, Short y Integer (que se tratan en Números y Cadenas).

El siguiente ejemplo de código, SwitchDemo, declara un int llamado "month" cuyo valor representa un mes. El código muestra el nombre del mes, basándose en el valor de "month", mediante la sentencia switch.

```
public class SwitchDemo {
    public static void main(String[] args) {
        int month;
        //Reading the value
        Scanner inputValue;
```

```
        inputValue = new Scanner(System.in);
        month = inputValue.nextInt();

        String monthString;
        switch (month) {
            case 1: monthString = "January";
                    break;
            case 2: monthString = "February";
                    break;
            case 3: monthString = "March";
                    break;
            case 4: monthString = "April";
                    break;
            case 5: monthString = "May";
                    break;
            case 6: monthString = "June";
                    break;
            case 7: monthString = "July";
                    break;
            case 8: monthString = "August";
                    break;
            case 9: monthString = "September";
                    break;
            case 10: monthString = "October";
                    break;
            case 11: monthString = "November";
                    break;
            case 12: monthString = "December";
                    break;
            default: monthString = "Invalid month";
                    break;
        }
        System.out.println(monthString);
    }
}
```

En este caso, agosto se imprime en la salida estándar.

El cuerpo de una sentencia switch se conoce como bloque switch. Una sentencia en el bloque switch puede etiquetarse con una o más etiquetas case o default. La sentencia switch evalúa su expresión y luego ejecuta todas las sentencias que siguen a la etiqueta case correspondiente.

También se puede mostrar el nombre del mes con sentencias if-then-else:

```
int month = 8;
if (month == 1) {
    System.out.println("January");
} else if (month == 2) {
    System.out.println("February");
}
... // and so on
```

La decisión de usar sentencias if-then-else o una sentencia switch se basa en la legibilidad y la expresión que la sentencia evalúa. Una sentencia if-then-else puede evaluar expresiones basándose en rangos de valores o condiciones, mientras que una sentencia switch evalúa expresiones basándose únicamente en un único entero, valor enumerado u objeto String.

Otro punto de interés es la sentencia break. Cada sentencia break finaliza la sentencia switch que la encierra. El flujo de control continúa con la primera sentencia después del bloque switch. Las sentencias break son necesarias porque, sin ellas, las sentencias de los bloques switch no se ejecutan: todas las sentencias posteriores a la etiqueta case correspondiente se ejecutan en secuencia, independientemente de la expresión de las etiquetas case subsiguientes, hasta que se encuentra una sentencia break. El programa SwitchDemoFallThrough muestra las sentencias de un bloque switch que no se ejecutan. El programa muestra el mes correspondiente al mes entero y los meses siguientes del año:

```
public class SwitchDemoFallThrough {  
    public static void main(String[] args) {  
  
        int month = 8;  
  
        switch (month) {  
            case 1: System.out.println("January");  
            case 2: System.out.println("February");  
            case 3: System.out.println("March");  
            case 4: System.out.println("April");  
            case 5: System.out.println("May");  
            case 6: System.out.println("June");  
            case 7: System.out.println("July");  
            case 8: System.out.println("August");  
            case 9: System.out.println("September");  
            case 10: System.out.println("October");  
            case 11: System.out.println("November");  
            case 12: System.out.println("December");  
                    break;  
            default: break;  
        }  
    }  
}
```

Esta es la salida del código:

```
August  
September  
October  
November  
December
```

Técnicamente, el break final no es necesario porque el flujo se desvía de la sentencia switch. Se recomienda usar un break para facilitar la modificación del código y reducir la probabilidad de errores. La sección predeterminada gestiona todos los valores que no se gestionan explícitamente en una de las secciones case.

El siguiente ejemplo de código, SwitchDemo2, muestra cómo una sentencia puede tener varias etiquetas case. El ejemplo de código calcula el número de días de un mes determinado:

```
class SwitchDemo2 {
    public static void main(String[] args) {

        int year = 2000;
        int numDays = 0;
        int month;
        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        month = inputValue.nextInt();

        switch (month) {
            case 1: case 3: case 5:
            case 7: case 8: case 10:
            case 12:
                numDays = 31;
                break;
            case 4: case 6:
            case 9: case 11:
                numDays = 30;
                break;
            case 2:
                if (((year % 4 == 0) &&
                    !(year % 100 == 0))
                    || (year % 400 == 0))
                    numDays = 29;
                else
                    numDays = 28;
                break;
            default:
                System.out.println("Invalid month.");
                break;
        }
        System.out.println("Number of Days = "
                           + numDays);
    }
}
```

Esta es la salida del código para mes = 2:

```
Number of Days = 29
```

Uso de cadenas en sentencias switch

En Java SE 7 y versiones posteriores, se puede usar un objeto String en la expresión de la sentencia switch. El siguiente ejemplo de código, StringSwitchDemo, muestra el número del mes basándose en el valor de la cadena denominada "month":

```
public class StringSwitchDemo {  
  
    public static int getMonthNumber(String month) {  
  
        int monthNumber = 0;  
  
        if (month == null) {  
            return monthNumber;  
        }  
  
        switch (month.toLowerCase()) {  
            case "january":  
                monthNumber = 1;  
                break;  
            case "february":  
                monthNumber = 2;  
                break;  
            case "march":  
                monthNumber = 3;  
                break;  
            case "april":  
                monthNumber = 4;  
                break;  
            case "may":  
                monthNumber = 5;  
                break;  
            case "june":  
                monthNumber = 6;  
                break;  
            case "july":  
                monthNumber = 7;  
                break;  
            case "august":  
                monthNumber = 8;  
                break;  
            case "september":  
                monthNumber = 9;  
                break;  
            case "october":  
                monthNumber = 10;  
                break;  
            case "november":  
                monthNumber = 11;  
                break;  
            case "december":  
                monthNumber = 12;  
                break;  
            default:
```

```
        monthNumber = 0;
        break;
    }

    return monthNumber;
}

public static void main(String[] args) {

    String month = "August";

    int returnedMonthNumber =
        StringSwitchDemo.getMonthNumber(month);

    if (returnedMonthNumber == 0) {
        System.out.println("Invalid month");
    } else {
        System.out.println(returnedMonthNumber);
    }
}
}
```

La salida de este código es 8.

El String de la expresión switch se compara con las expresiones asociadas a cada etiqueta de caso como si se usara el método `String.equals`. Para que el ejemplo `StringSwitchDemo` acepte cualquier mes, independientemente de si se trata de mayúsculas o minúsculas, el mes se convierte a minúsculas (con el método `toLowerCase`) y todas las cadenas asociadas a las etiquetas de caso se convierten a minúsculas.

Nota: Este ejemplo comprueba si la expresión de la sentencia switch es nula. Asegúrese de que la expresión de cualquier sentencia switch no sea nula para evitar que se genere una `NullPointerException`.

3.4.4. while y do-while

La instrucción `while` ejecuta continuamente un bloque de instrucciones mientras una condición específica sea verdadera. Su sintaxis se puede expresar como:

```
while (expression) {
    statement(s)
}
```

La instrucción `while` evalúa la expresión, que debe devolver un valor booleano. Si la expresión se evalúa como verdadera, la instrucción `while` ejecuta la(s) instrucción(es) en el bloque `while`. La instrucción `while` continúa probando la expresión y ejecutando su bloque hasta que se evalúa como falsa. Usar la instrucción `while` para imprimir los valores del 1 al 10 se puede lograr como en el siguiente programa `WhileDemo`:

```
class WhileDemo {
```

```
        public static void main(String[] args){
            int count = 1;
            while (count < 11) {
                System.out.println("Count is: " + count);
                count++;
            }
        }
    }
```

Puedes implementar un bucle infinito utilizando la declaración while de la siguiente manera:

```
while (true){
    // your code goes here
}
```

El lenguaje de programación Java también proporciona una declaración do-while, que puede expresarse de la siguiente manera:

```
do {
    statement(s)
} while (expression);
```

La diferencia entre do-while y while radica en que do-while evalúa su expresión al final del bucle, en lugar de al principio. Por lo tanto, las sentencias dentro del bloque do siempre se ejecutan al menos una vez, como se muestra en el siguiente programa DoWhileDemo:

```
class DoWhileDemo {
    public static void main(String[] args){
        int count = 1;
        do {
            System.out.println("Count is: " + count);
            count++;
        } while (count < 11);
    }
}
```

3.4.5. for

La sentencia for proporciona una forma compacta de iterar sobre un rango de valores. Los programadores suelen denominarla "bucle for" debido a la forma en que se repite hasta que se cumple una condición específica. La forma general de la sentencia for se puede expresar de la siguiente manera:

```
for (initialization; termination; increment) {
    statement(s)
}
```

Al usar esta versión de la instrucción for, tenga en cuenta que:

- La expresión de inicialización inicializa el bucle; se ejecuta una sola vez, al comenzar.

- Cuando la expresión de terminación se evalúa como falsa, el bucle termina.
- La expresión de incremento se invoca después de cada iteración del bucle; es perfectamente aceptable que esta expresión incremente o decremente un valor.

El siguiente programa, ForDemo, utiliza la forma general de la instrucción for para imprimir los números del 1 al 10 en la salida estándar:

```
class ForDemo {
    public static void main(String[] args){
        for(int i = 1; i < 11; i++){
            System.out.println("Count is: " + i);
        }
    }
}
```

La salida de este programa es:

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
Count is: 6
Count is: 7
Count is: 8
Count is: 9
Count is: 10
```

Observa cómo el código declara una variable dentro de la expresión de inicialización. El alcance de esta variable se extiende desde su declaración hasta el final del bloque regido por la sentencia for, por lo que también puede usarse en las expresiones de terminación e incremento. Si la variable que controla una sentencia for no se necesita fuera del bucle, es mejor declararla en la expresión de inicialización. Los nombres i, j y k se usan a menudo para controlar bucles for; declararlos dentro de la expresión de inicialización limita su duración y reduce los errores.

Las tres expresiones del bucle for son opcionales; un bucle infinito se puede crear de la siguiente manera:

```
// infinite loop
for ( ; ; ) {
    // your code goes here
}
```

Para saber más

La instrucción for también tiene otra forma diseñada para la iteración a través de colecciones y arrays. Esta forma a veces se denomina instrucción for mejorada y puede

usarse para que los bucles sean más compactos y fáciles de leer. Para demostrarlo, considere el siguiente array, que contiene los números del 1 al 10:

```
int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

El siguiente programa, EnhancedForDemo, utiliza el método for mejorado para recorrer el array:

```
class EnhancedForDemo {
    public static void main(String[] args){
        int[] numbers =
            {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
        for (int item: numbers) {
            System.out.println("Count is: " + item);
        }
    }
}
```

En este ejemplo, la variable item contiene el valor actual del array numbers. La salida de este programa es la misma que antes.

3.4.6. Instrucciones break, continue y return

break

La sentencia break tiene dos formas: etiquetada y no etiquetada. Ya viste la forma no etiquetada en la explicación anterior de la sentencia switch. También puedes usar una sentencia break no etiquetada para terminar un bucle for, while o do-while, como se muestra en el siguiente programa BreakDemo:

```
import java.util.Scanner;
class BreakDemo {
    public static void main(String[] args) {
        boolean prime = true;
        int n;
        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        n = inputValue.nextInt();

        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                prime = false;
                break;
            }
        }
        if (prime) {
            System.out.println("Prime");
        } else {
            System.out.println("NOT Prime");
        }
    }
}
```

Este programa muestra si un número es primo o no. En el momento en que se detecta que no es primo, no es necesario seguir iterando en el for, por lo que se cierra con la sentencia break.

Una sentencia break sin etiquetar finaliza la sentencia switch, for, while o do-while más interna, pero una sentencia break etiquetada finaliza una sentencia externa. El siguiente programa, BreakWithLabelDemo, es similar al anterior, pero utiliza bucles for anidados para buscar un valor en una matriz bidimensional. Cuando se encuentra el valor, una sentencia break etiquetada finaliza el bucle for externo (etiquetado como "búsqueda").

```
import java.util.Scanner;
class BreakWithLabelDemo {
    public static void main(String[] args) {
        //Reading the value
        Scanner inputValue;
        inputValue = new Scanner(System.in);
        n = inputValue.nextInt();

        search:
        for (i = 0; i < 200; i++) {
            for (j = 0; j < 300; j++) {
                if (i * j > n) {
                    break search;
                } else {
                    System.out.println(i + ":" + j);
                }
            }
        }
    }
}
```

La sentencia break finaliza la sentencia etiquetada; no transfiere el flujo de control a la etiqueta. El flujo de control se transfiere a la sentencia inmediatamente posterior a la sentencia etiquetada (terminada).

continue

La instrucción continue omite la iteración actual de un bucle for, while o do-while. La forma sin etiquetar salta al final del cuerpo del bucle más interno y evalúa la expresión booleana que lo controla. El siguiente programa, ContinueDemo, recorre una cadena, contando las ocurrencias de la letra "p". Si el carácter actual no es una p, la instrucción continue omite el resto del bucle y continúa con el siguiente carácter. Si es una "p", el programa incrementa el recuento de letras.

```
class ContinueDemo {
    public static void main(String[] args) {

        String searchMe = "peter piper picked a " +
            "peck of pickled peppers";
```

```
int max = searchMe.length();
int numPs = 0;

for (int i = 0; i < max; i++) {
    // interested only in p's
    if (searchMe.charAt(i) != 'p') {
        continue;
    }

    // process p's
    numPs++;
}
System.out.println("Found " + numPs +
    " p's in the string.");
}
```

Aquí está el resultado de este programa:

Found 9 p's in the string.

Para ver este efecto con mayor claridad, intenta eliminar la instrucción "continue" y vuelve a compilar. Al volver a ejecutar el programa, el conteo será incorrecto, indicando que encontró 35 p en lugar de 9.

return

La instrucción return sale del método actual y el flujo de control regresa al punto donde se invocó el método. La instrucción return tiene dos formas: una que devuelve un valor y otra que no. Para devolver un valor, simplemente coloque el valor (o una expresión que lo calcule) después de la palabra clave return.

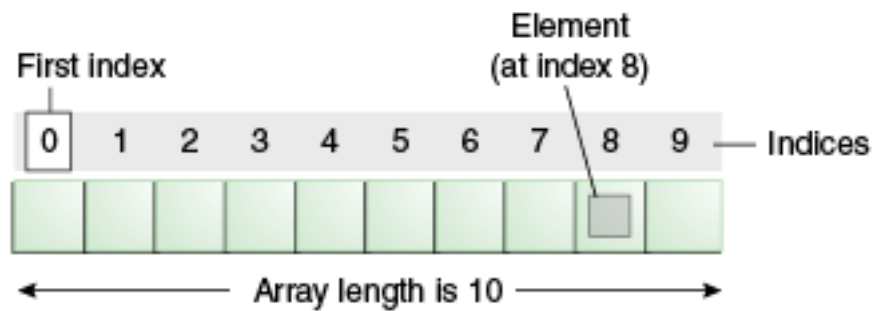
```
return ++count;
```

El tipo de dato del valor devuelto debe coincidir con el tipo del valor de retorno declarado del método. Cuando un método se declara nulo, utilice la forma de retorno que no devuelve un valor.

```
return;
```

3.5. Arrays

Un array es un objeto contenedor que contiene un número fijo de valores de un solo tipo. Su longitud se establece al crearlo. Tras su creación, su longitud es fija. Ya has visto un ejemplo de arrays en el método main de la aplicación "¡Hola Mundo!". Esta sección los analiza con más detalle.



Cada elemento de un array se denomina elemento, y se accede a cada uno de ellos mediante su índice numérico. Como se muestra en la ilustración anterior, la numeración comienza en 0. Por ejemplo, el noveno elemento se accedería en el índice 8.

El siguiente programa, `ArrayDemo.java`, crea un array de números enteros, coloca algunos valores en el array e imprime cada valor en la salida estándar.

```
class ArrayDemo {
    public static void main(String[] args) {
        // declares an array of integers
        int[] anArray;

        // allocates memory for 10 integers
        anArray = new int[10];

        // initialize first element
        anArray[0] = 100;
        // initialize second element
        anArray[1] = 200;
        // and so forth
        anArray[2] = 300;
        anArray[3] = 400;
        anArray[4] = 500;
        anArray[5] = 600;
        anArray[6] = 700;
        anArray[7] = 800;
        anArray[8] = 900;
        anArray[9] = 1000;

        System.out.println("Element at index 0: "
            + anArray[0]);
        System.out.println("Element at index 1: "
            + anArray[1]);
        System.out.println("Element at index 2: "
            + anArray[2]);
        System.out.println("Element at index 3: "
            + anArray[3]);
        System.out.println("Element at index 4: "
            + anArray[4]);
        System.out.println("Element at index 5: "
```

```
        + anArray[5]);  
    System.out.println("Element at index 6: "  
        + anArray[6]);  
    System.out.println("Element at index 7: "  
        + anArray[7]);  
    System.out.println("Element at index 8: "  
        + anArray[8]);  
    System.out.println("Element at index 9: "  
        + anArray[9]);  
    }  
}
```

La salida de este programa es:

```
Element at index 0: 100  
Element at index 1: 200  
Element at index 2: 300  
Element at index 3: 400  
Element at index 4: 500  
Element at index 5: 600  
Element at index 6: 700  
Element at index 7: 800  
Element at index 8: 900  
Element at index 9: 1000
```

En una situación de programación real, probablemente usaría una de las construcciones de bucle compatibles para iterar por cada elemento del array, en lugar de escribir cada línea individualmente como en el ejemplo anterior. Sin embargo, el ejemplo ilustra claramente la sintaxis del array. Aprenderá sobre las diversas construcciones de bucle (for, while y do-while) en la sección Control de flujo.

3.5.1. Declarando un Array

El programa anterior declara un array (llamado anArray) con la siguiente línea de código:

```
// declares an array of integers  
int[] anArray;
```

Al igual que las declaraciones de variables de otros tipos, la declaración de un array tiene dos componentes: el tipo y el nombre del array. El tipo de un array se escribe como `type[]`, donde `type` es el tipo de dato de los elementos que contiene; los corchetes son símbolos especiales que indican que esta variable contiene un array. El tamaño del array no forma parte de su tipo (por eso los corchetes están vacíos). El nombre de un array puede ser cualquiera, siempre que siga las reglas y convenciones explicadas en la sección de nombres. Al igual que con las variables de otros tipos, la declaración no crea un array; simplemente indica al compilador que esta variable contendrá un array del tipo especificado.

De manera similar, puedes declarar arrays de otros tipos:

```
byte[]  anArrayOfBytes;
short[] anArrayOfShorts;
long[]  anArrayOfLongs;
float[] anArrayOfFloats;
double[] anArrayOfDoubles;
boolean[] anArrayOfBooleans;
char[]  anArrayOfChars;
String[] anArrayOfStrings;
```

También puedes colocar los corchetes después del nombre del array:

```
// this form is discouraged
float anArrayOfFloats[];
```

Sin embargo, la convención desaconseja esta forma; los corchetes identifican el tipo de array y deben aparecer con la designación del tipo.

3.5.2. Creación, inicialización y acceso a un array.

Una forma de crear un array es con el operador `new`. La siguiente instrucción del programa `ArrayDemo` asigna un array con memoria suficiente para 10 elementos enteros y lo asigna a la variable `anArray`.

```
// create an array of integers
anArray = new int[10];
```

Si falta esta declaración, el compilador imprime un error como el siguiente y la compilación falla:

```
ArrayDemo.java:4: Variable anArray may not have been
initialized.
```

Las siguientes líneas asignan valores a cada elemento del array:

```
anArray[0] = 100; // initialize first element
anArray[1] = 200; // initialize second element
anArray[2] = 300; // and so forth
```

Se accede a cada elemento del array mediante su índice numérico:

```
System.out.println("Element 1 at index 0: " + anArray[0]);
System.out.println("Element 2 at index 1: " + anArray[1]);
System.out.println("Element 3 at index 2: " + anArray[2]);
```

Alternativamente, puede utilizar la sintaxis de acceso directo para crear e inicializar un array:

```
int[] anArray = {100, 200, 300, 400, 500, 600,
                 700, 800, 900, 1000 };
```

Aquí, la longitud del array se determina por el número de valores proporcionados entre llaves y separados por comas.

También se puede declarar un array de arrays (también conocido como array multidimensional o matriz) utilizando dos o más conjuntos de corchetes, como los nombres `String[][]`. Por lo tanto, se debe acceder a cada elemento mediante un número correspondiente de valores de índice.

En el lenguaje de programación Java, un array multidimensional es un array cuyos componentes son, a su vez, arrays. Esto difiere de los arrays en C o Fortran. Como consecuencia, se permite que las filas varíen en longitud, como se muestra en el siguiente programa `MultiArrayDemo.java`:

```
class MultiDimArrayDemo {
    public static void main(String[] args) {
        String[][] names = {
            {"Mr. ", "Mrs. ", "Ms. "},
            {"Smith", "Jones"}
        };
        // Mr. Smith
        System.out.println(names[0][0] + names[1][0]);
        // Ms. Jones
        System.out.println(names[0][2] + names[1][1]);
    }
}
```

La salida de este programa es:

```
Mr. Smith
Ms. Jones
```

Finalmente, puedes usar la propiedad `length` para determinar el tamaño de cualquier array. El siguiente código imprime el tamaño del array en la salida estándar:

```
System.out.println(anArray.length);
```

3.5.3. Copiando Arrays

La clase `System` tiene un método `arraycopy()` que puede usar para copiar eficientemente datos de un array a otro:

```
public static void arraycopy(Object src, int srcPos,
                             Object dest, int destPos, int length)
```

Los dos argumentos "Object" especifican el array del que se copiará y el array al que se copiará. Los tres argumentos "int" especifican la posición inicial en el array de origen, la posición inicial en el array de destino y el número de elementos del array que se copiarán.

El siguiente programa, ArrayCopyDemo.java, declara un array de elementos char, que forman la palabra "decaffeinated". Utiliza el método System.arraycopy() para copiar una subsecuencia de componentes del array en un segundo array:

```
class ArrayCopyDemo {
    public static void main(String[] args) {
        char[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f',
                           'e', 'i', 'n', 'a', 't', 'e', 'd' };
        char[] copyTo = new char[7];

        System.arraycopy(copyFrom, 2, copyTo, 0, 7);
        System.out.println(new String(copyTo));
    }
}
```

La salida de este programa es:

```
caffein
```

3.5.4. Manipulación de arrays

Los arrays son un concepto potente y útil en programación. Java SE proporciona métodos para realizar algunas de las manipulaciones más comunes relacionadas con los arrays. Por ejemplo, el ejemplo ArrayCopyDemo utiliza el método arraycopy() de la clase System en lugar de iterar manualmente los elementos del array de origen y colocar cada uno en el array de destino. Esto se realiza en segundo plano, lo que permite al desarrollador usar una sola línea de código para llamar al método.

Para mayor comodidad, Java SE proporciona varios métodos para realizar manipulaciones de arrays (tareas comunes, como copiar, ordenar y buscar arrays) en la clase java.util.Arrays. Por ejemplo, el ejemplo anterior se puede modificar para usar el método CopyOfRange() de la clase java.util.Arrays, como se puede ver en el ejemplo ArrayCopyOfDemo. La diferencia radica en que usar el método CopyOfRange() no requiere crear el array de destino antes de llamarlo, ya que este lo devuelve:

```
import java.util.Arrays;
class ArrayCopyOfDemo {
    public static void main(String[] args) {

        char[] copyFrom = {'d', 'e', 'c', 'a', 'f', 'f', 'e',
                           'i', 'n', 'a', 't', 'e', 'd'};
        char[] copyTo = Arrays.copyOfRange(copyFrom, 2, 9);

        System.out.println(new String(copyTo));
    }
}
```

```
}

```

Como puedes ver, la salida de este programa es la misma (caffein), aunque requiere menos líneas de código.

Otras operaciones útiles que ofrecen los métodos de la clase `java.util.Arrays` son:

- Buscar un valor específico en un array para obtener su índice (método `binarySearch()`).
- Comparar dos arrays para determinar si son iguales (método `equals()`).
- Rellenar un array para colocar un valor específico en cada índice (método `fill()`).
- Ordenar un array en orden ascendente. Esto se puede realizar secuencialmente, mediante el método `sort()`, o simultáneamente, mediante el método `parallelSort()`, introducido en Java SE 8. La ordenación paralela de arrays grandes en sistemas multiprocesador es más rápida que la ordenación secuencial.

3.5.5. Ejercicios con Arrays

1. Escribe un programa que lea 10 números y los muestre en orden inverso.
2. Escribe un programa que lea 10 números y luego invierta el orden intercambiando elementos, y escribe el resultado comenzando por el primer elemento del nuevo array ordenado.
3. Escribe un programa que lea un DNI (sin la letra) y luego calcule la letra, sabiendo que esta se calcula de esta manera:

Toma el DNI como un número y calcule el resto de la división entre 23, y asigna la letra según el resto utilizando la siguiente tabla:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
T	R	W	A	G	M	Y	F	P	D	X	B	N	J	Z	S	Q	V	H	L	C	K	E

4. Escribe un programa que cree una matriz de 10 x 10 elementos. Inicializa sus elementos a 1 y luego establece a 8 los elementos (0, 4), (2, 6), (3, 1) y (8, 6). Y luego la imprimes así:

```

      |   |   |   |   8   |   |   |   |   | |
      |   |   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   8   |   |   |
      |   8   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |   |   |
      |   |   |   |   |   |   |   |   |   |

```

```
| | | | | | 8 | | |
| | | | | | | | | |
```

5. Modifica el programa anterior para mostrar cuántas filas y columnas tienen todos los elementos a 1.

6. Escribe un programa que calcule la moda (el elemento más frecuente) de una lista de 20 caracteres (inicialice el array de caracteres en la declaración del array como una lista de caracteres).

7. Escribe un programa que simule el juego "Batalla Naval" sobre un tablero de 8x8. La computadora colocará 10 barcos al azar (de una sola celda).

El usuario introducirá una coordenada (primera letra y luego número) y el programa escribirá el tablero completo cada vez, con los barcos hundidos (X) y los disparos fallidos (círculos). Mostrará el contador de disparos y el número de barcos hundidos.

Ejemplo:

SHOTS: 12

SUNK SHIPS: 3

	1	2	3	4	5	6	7	8
A								
B		O		O		O		
C		O					O	
D	X	O						
E				X	O		O	
F	O			O				
G	O				O		X	
H								O

Enter row (Letter): C

Enter column (Number): 2

3.5.6. Ordenando Arrays

Existen métodos de ordenación en la clase Arrays que permiten ordenar un array, como Arrays.sort(array), que realiza esta tarea automáticamente. Sin embargo, en ocasiones, cuando los elementos son objetos y necesitamos compararlos de forma específica, puede ser necesario implementar nuestro propio método de ordenación. Veremos un par de algoritmos de ordenación para practicar la manipulación de arrays.

Select-Sort

En este método, buscamos el elemento más pequeño y lo insertamos en la primera posición del array. Luego, con el segundo elemento, realizamos el mismo procedimiento para el resto de los elementos.

Puedes ver una representación del algoritmo en este video:

<https://www.youtube.com/watch?v=Ns4TPTC8whw>

Ejercicio

Implementa el algoritmo Select-Sort en Java. Pruébalo con un array de cadenas inicializado en el programa (sin leerlas con el teclado).

Bubble-Sort

En este método, se comparan el primer y el segundo elemento. Si el primero es mayor que el segundo, intercambian posiciones. Luego, se comparan el segundo y el tercero, y se procede de la misma manera hasta llegar al final del array. El último elemento se ordenará. En la siguiente iteración, se sigue el mismo procedimiento, pero se llega solo al elemento n-1. En la siguiente iteración, se repiten n-2 hasta llegar al elemento 1.

Este procedimiento suele obtener un array ordenado antes de llegar a la iteración del elemento 1, por lo que se puede mejorar detectando si no se produjo ningún intercambio. En ese caso, el array ya está ordenado.

Puedes ver una representación del algoritmo en este video:

<http://www.youtube.com/watch?v=lyZQPjUT5B4&list=PLP2eLJiSCJbAclmdE6sqx30XphnZ4Rjo>

Ejercicio

Implementa el algoritmo Bubble-Sort en Java. Pruébalo con un array de cadenas inicializadas en el programa (sin leerlas con el teclado).

3.5.7. Acceder a elementos de un array con un bucle For

La instrucción for también tiene otra forma diseñada para la iteración a través de colecciones y arrays. Esta forma a veces se denomina instrucción for mejorada y puede usarse para que los bucles sean más compactos y fáciles de leer. Para demostrarlo, considere el siguiente array, que contiene los números del 1 al 10:

```
int[] numbers = {1,2,3,4,5,6,7,8,9,10};
```

El siguiente programa, EnhancedForDemo, utiliza el método for mejorado para recorrer el array:

```
class EnhancedForDemo {  
    public static void main(String[] args){  
        int[] numbers =
```

```
        {1,2,3,4,5,6,7,8,9,10};  
    for (int item: numbers) {  
        System.out.println("Count is: " + item);  
    }  
}
```

En este ejemplo, la variable `item` contiene el valor actual del array de números. El resultado de este programa es el mismo que antes:

Recomendamos usar esta forma de la instrucción `for` en lugar de la forma general siempre que sea posible.